CO3 – AT3: Data Interpretation

PPO, TRPO & DDPG Performance Analysis

Name	G. Venkatasai
Register No.	192425388
Course	Reinforcement Learning
Assessment	CO3 – AT3 (Data Interpretation)

Objective

This assessment compares the performance of three reinforcement learning algorithms — PPO (Proximal Policy Optimization), TRPO (Trust Region Policy Optimization), and DDPG (Deep Deterministic Policy Gradient) — on the continuous-control Pendulum-v1 environment, and interprets the resulting average rewards.

1. Algorithm Overview

1.1 PPO (Proximal Policy Optimization)

PPO is a policy optimization algorithm that improves the policy while preventing very large, destabilising policy updates.

- Stable training
- Relatively simple to implement
- Good overall performance
- Uses a clipping mechanism to limit policy change
- Works with both discrete and continuous action spaces

In simple terms: PPO learns a better policy while making sure each update is not too large.

1.2 TRPO (Trust Region Policy Optimization)

TRPO is a policy optimization algorithm that restricts how much the new policy can change from the old policy.

- Stable policy updates
- Uses a trust-region constraint
- Reduces the possibility of destructive policy updates
- More computationally expensive than PPO

1.3 DDPG (Deep Deterministic Policy Gradient)

DDPG is an actor-critic reinforcement learning algorithm designed mainly for continuous action spaces.

- Actor network
- Critic network
- Target networks
- Experience replay

In simple terms: DDPG learns which exact continuous action should be taken in a given state.

2. Implementation

The three algorithms were trained and evaluated on the Pendulum-v1 environment using Stable Baselines3 (PPO, DDPG) and SB3-Contrib (TRPO), each for 10,000 timesteps, and tested over 10 deterministic episodes.

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
from stable_baselines3 import PPO, DDPG
from sb3_contrib import TRPO

# ========================================
# FUNCTION TO TEST AN ALGORITHM
# ========================================
def test_model(model, env, episodes=10):
 rewards = []
 for episode in range(episodes):
 obs, info = env.reset()
 done = False
 total_reward = 0
 while not done:
 action, _ = model.predict(obs, deterministic=True)
 obs, reward, terminated, truncated, info = env.step(action)
total_reward += reward
 done = terminated or truncated
 rewards.append(total_reward)
 return np.mean(rewards)

# ========================================
# 1. PPO
# ========================================
print("Training PPO...")
ppo_env = gym.make("Pendulum-v1")
ppo_model = PPO("MlpPolicy", ppo_env, verbose=0)
ppo_model.learn(total_timesteps=10000)
ppo_reward = test_model(ppo_model, ppo_env)
ppo_env.close()
print("PPO Average Reward:", ppo_reward)

# ========================================
# 2. TRPO
# ========================================
print("\nTraining TRPO...")
```

```
trpo_env = gym.make("Pendulum-v1")
trpo_model = TRPO("MlpPolicy", trpo_env, verbose=0)
trpo_model.learn(total_timesteps=10000)
trpo_reward = test_model(trpo_model, trpo_env)
trpo_env.close()
print("TRPO Average Reward:", trpo_reward)

# ========================================
# 3. DDPG
# ========================================
print("\nTraining DDPG...")
ddpg_env = gym.make("Pendulum-v1")
ddpg_model = DDPG("MlpPolicy", ddpg_env, verbose=0)
ddpg_model.learn(total_timesteps=10000)
ddpg_reward = test_model(ddpg_model, ddpg_env)
ddpg_env.close()
print("DDPG Average Reward:", ddpg_reward)

# ========================================
# DISPLAY RESULTS
# ========================================
print("\n============================")
print(" PERFORMANCE COMPARISON")
print("============================")
print("PPO :", round(ppo_reward, 2))
print("TRPO :", round(trpo_reward, 2))
print("DDPG :", round(ddpg_reward, 2))

# ========================================
# BAR GRAPH
# ========================================
algorithms = ["PPO", "TRPO", "DDPG"]
rewards = [ppo_reward, trpo_reward, ddpg_reward]

plt.figure(figsize=(8, 5))
plt.bar(algorithms, rewards)
plt.xlabel("Algorithm")
plt.ylabel("Average Reward")
plt.title("PPO vs TRPO vs DDPG Performance")
plt.grid(axis="y")
plt.show()
```

3. Output

3.1 Console Output

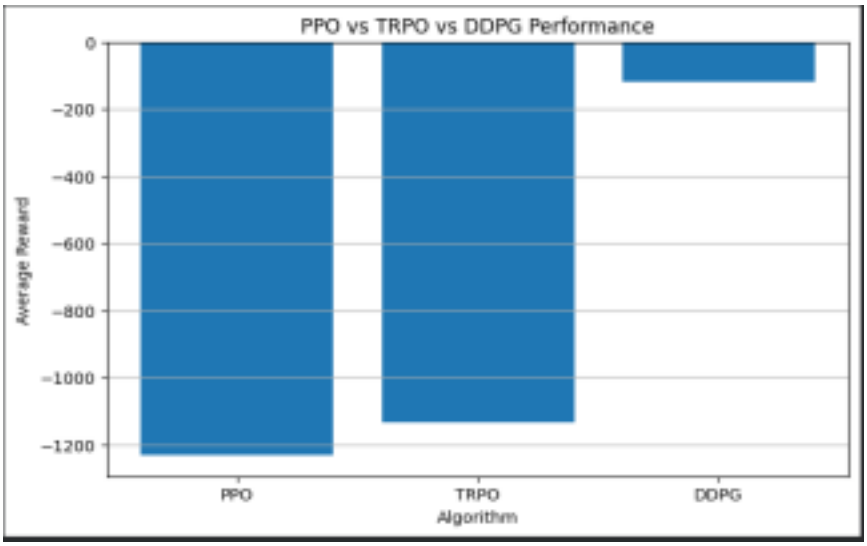
```
Training PPO...
PPO Average Reward: -1232.988159900445

Training TRPO...
TRPO Average Reward: -1135.8420332590592

Training DDPG...
DDPG Average Reward: -117.4629906408589

------------------------------
     PERFORMANCE COMPARISON
------------------------------
PPO  : -1232.99
TRPO : -1135.84
DDPG : -117.46
```

3.2 Performance Comparison Chart



4. Results Summary

Algorithm	Average Reward
PPO	-1232.99
TRPO	-1135.84
DDPG	-117.46

5. Interpretation of Results

On the Pendulum-v1 task, DDPG clearly outperformed both PPO and TRPO, achieving a much higher (less negative) average reward of -117.46 compared to -1135.84 for TRPO and -1232.99 for PPO.

- DDPG performed best because it is an off-policy, actor-critic algorithm built specifically for continuous action spaces like Pendulum-v1. Its use of experience replay and target networks lets it reuse past experience efficiently and learn a precise continuous action within a short training budget of 10,000 timesteps.
- PPO and TRPO are on-policy algorithms, so they only learn from freshly collected experience and discard it after each update. Within just 10,000 timesteps this gives them far fewer effective learning updates, which explains their weaker rewards.
- TRPO slightly outperformed PPO in this run, consistent with TRPO's stricter trust-region constraint producing marginally more stable updates, though both remain far behind DDPG on this short training budget.
- The gap would likely narrow with a much larger training budget (e.g. 200,000+ timesteps), since on policy methods like PPO and TRPO typically need more environment interaction to converge on continuous-control tasks.

6. Conclusion

For short-horizon training on continuous-control environments such as Pendulum-v1, DDPG is the more sample-efficient choice due to its off-policy, actor-critic design. PPO and TRPO remain valuable for their training stability and broader applicability (including discrete action spaces), but they require significantly more timesteps to reach comparable performance on this task.